HackenProof

Expert crowdsourced security provider

NEAR Intents: Bridges

Report name:

Github Workflow injection leading to Secrets leak (DOCKERHUB_TOKEN, NEAR/MPC repo write access)

PREPARED BY
HackenProof

REPORT CREATED
05.04.2026

REPORT ID
NIB-262

SEVERITY
High

AUTHOR
natsec

VULNERABILITY
Critically Sensitive Data

TARGET

`https://github.com/near/mpc`

VULNERABILITY DETAILS

The NEAR/MPC workflow claude-pr-review.yml is vulnerable to an environment variable injection attack.

This leaks

- Claude Code Action Github authentication token (commit:write, pull_requests:write, issues:write, workflow:write)
- This allows you to read Github repo secrets such as DOCKERHUB_TOKEN
- Also allows for creating fake backdoored releases and editing existing releases + digest

Vulnerability details and POC

To execute the POC you need to setup a new private repository with secrets, the readme has instructions. You can also watch the video.

Vulnerability

- An attacker creates a regular fork pull request to NEAR/MPC repo (no auth required)
- The attacker creates a comment calling the claude-pr-review workflow "@claude review this code + ${payload}"
- This gives attacker code execution inside the workflow VM.
- Because the workflow was triggered by the **issue_comment** event, the workflow VM has full access to all secrets and the OIDC token which allows for escalating to full repo access

This can also be done on any open PR, the PR does not have to be created by the attacker.

Root Cause

1. Environment variable injection

claude-pr-review.yml writes untrusted external PR comment content into a $GITHUB_ENV heredoc:

```
{
    echo 'REVIEW_PROMPT<<PROMPT_EOF'
    cat /tmp/pr_comments_context.txt   # attacker-controlled
    echo 'PROMPT_EOF'
} >> "$GITHUB_ENV"
```

A comment body containing PROMPT_EOF on its own line closes the heredoc early, this allows for setting arbitrary environment variable KEY=VALUE pairs into the workflow VM.

Normal comment body (@claude review this code):

```
REVIEW_PROMPT<<PROMPT_EOF
@claude review this code       # goes into REVIEW_PROMPT value
PROMPT_EOF                     # closes REVIEW_PROMPT
```

Malicious comment body containing PROMPT_EOF on its own line:

```
REVIEW_PROMPT<<PROMPT_EOF
nvm, closing this
PROMPT_EOF                     # closes REVIEW_PROMPT early (injection point)
BUN_OPTIONS=-e <payload>       # now outside any heredoc can set any environment v
TRASH<<PROMPT_EOF              # absorbs the remaining content
...rest of file...
PROMPT_EOF
```

The injected payload is hidden inside a HTML block which makes it invisible, an attacker could also edit the comment.

2. Code execution inside workflow VM from environment variable injection

Using the environment variable BUN_OPTIONS, you can set options to be passed to Bun on every call to Bun. Bun supports "--eval/-e" to execute code passed as CLI argument to Bun. This gives code execution escalation from environment variable injection. Bun is called by the Claude Code Action which causes the code to be executed by Bun.

With this payload you can read any secrets/environment variables that the workflow VM has access to.

3. issue_comment trigger bypasses normal fork PR protection

Normally fork PRs triggered workflows have no access to secrets, however the issue_comment triggered events run in the base repository context with full secrets and OIDC access.

4. Payload

Within this workflow VM, the attacker has access to the Claude Code Action Github token, which has write access to

- Commits
- Workflows (To get this requires escalating access using ACTIONS_ID_TOKEN_REQUEST_TOKEN)
- Releases
- Tags
- Pull requests
- Issues

With the workflow write and commit write access, the attacker can create a new workflow that reads any Github secret and sends it to the attacker. This is because the attacker can write tags using the app token to trigger workflow execution.

The POC does all this and writes all the secrets + tokens as a comment response to the pull request, an attacker would extract it using curl or similar.

Payload steps

Mints OIDC JWT via ACTIONS_ID_TOKEN_REQUEST_URL which is available as an environment variable inside the workflow VM

Exchanges OIDC for App installation token (ghs_*) via https://api.anthropic.com/api/github/github-app-token-exchange, this grants the following permissions on the NEAR/MPC repo:

```
contents:write
    - Push any commit to non branch protected branches
    - Create branches
    - Create tags
    - Create releases, edit releases
    - Potential attack:
        - Replace release binaries/.wasm/contracts on existing releases,
        - Change the SHA256 digest inside the release notes
        - Use DOCKERHUB_TOKEN to also update Dockerhub image

workflows:write
    - Create new workflows to access Github repo secrets
    - Gives you access to all secrets

issues:write
pull_requests:write
repository hooks:write
```

Tag injection (workflows:write):

pushes ci-helper.yml + tag.

tag-triggered workflow reads (DOCKERHUB_TOKEN secret, any other secrets) and posts it as a PR comment

Removes eyes reaction by Claude from trigger comment (using issues:write)

Pushes cleanup.yml declaring actions:write, cleanup workflow deletes the logs of the attack run, ci-helper run, and itself

Calls process.exit(0) before run.ts starts, stops Claude from doing code review and commenting.

Very stealthy

- No action history
  - Deleted all logs
- No indicators on pull request
  - No claude code activity, no reactions on comment
  - Trigger comment shows only "nvm, closing this"
  - Payload injection not visible on Github frontend, comment can also be edited and payload injection removed
- Only visible through audit log.

Impact i was able to verify in my own repo with POC

- secrets.DOCKERHUB_TOKEN

  - Scope: publish any image to nearone/mpc-node, nearone/mpc-launcher, nearone/mpc-node-gcp

- Claude Code Action app authentication token

  - Scope: Attacker has same write permissions as Claude App permissions: contents (commit), pull_requests, issue, workflows.
  - Can push directly to non-protected branches for example, create releases (using workflows), change releases, create tags, create workflows to steal all Github repo and Near org scoped secrets.

- workflow: writes

  - Scope: Allows reading any org scoped Github secret by creating a one off workflow

- ACTIONS_ID_TOKEN_REQUEST_TOKEN and ACTIONS_ID_TOKEN_REQUEST_URL

  - Scope: These are used by Github to authenticate CI/CD workflows to external services

- ANTHROPIC_API_KEY

  - Billing DOS

VALIDATION STEPS

I have create a proof of concept for this vulnerability. To execute it you need to create a private Github repository and enter secrets. The readme has instructions, you can also watch the video.

I also wrote some mitigation and patching recomendations however i could not fit it in the previous step so i put it here.

Mitigation and patching

Patching the vulnerability

```
https://github.com/near/mpc/blob/main/.github/workflows/claude-pr-review.yml
https://github.com/near/mpc/blob/main/.github/workflows/pr-title-lint.yml
```

pr-title-lint.yml also has a similar vulnerability.

I think simplest is to disable and remove the workflows for now.

For fixing the injection vulnerability add a random delimiter which the attacker cannot predict.

https://codeql.github.com/codeql-query-help/actions/actions-envvar-injection-critical/

claude-pr-review.yml (lines 42–61):

```
      - name: Build review prompt
        id: build-prompt
        run: |
          UUID=$(uuidgen)
          {
            echo "REVIEW_PROMPT<<EOF$UUID"
            echo "EOF$UUID"
          } >> "$GITHUB_ENV"
```

pr-title-lint.yml (lines 54–66):

```
        run: |
          FILES=$(gh pr diff "$PR_NUMBER" --name-only)
          UUID=$(uuidgen)
          {
            echo "TITLE_CHECK_PROMPT<<EOF$UUID"
```

```
            echo "EOF$UUID"
          } >> "$GITHUB_ENV"
```

Rotating keys and mitigating potential attacks

Check audit logs for any indications of this attack having been executed previously.

For the pull request title injection, go through all pull requests on near/mpc and check that edits + original does not contain any env injection.

To check for pull request titles containing PROMPT_EOF

```
gh api "repos/near/mpc/pulls?state=all&per_page=100" --paginate --jq '.[] | select
```

Gives no results currently.

To check for edited pull requests title containing PROMPT_EOF

```
gh api "repos/near/mpc/pulls?state=all&per_page=100" --paginate --jq '.[].number'
while read num; do gh api "repos/near/mpc/issues/$num/timeline" --paginate --jq --
```

Gives no result currently.

For the pull request comment injection it is harder because there is no relaible way to read issue comment state as it can be deleted and edited, only viable way is workflow logs.

List all actors who triggered Claude Code Review via comment, check for unknown external actors

```
gh api "repos/near/mpc/actions/workflows/232540002/runs?per_page=100" --paginate -
```

Find deleted Claude Code Review runs, stealthy attacks leave this trace. Requires audit access

```
gh api "orgs/near/audit-log?phrase=action:workflows.delete_workflow_run+repo:near/
```

There be ways to bypass these indicators so i think it is a good idea to rotate keys anyway.

Recommended actions:

Rotate Docker Hub token for NEARONE user

Rotate ANTHROPIC_API key

Investigate current production MPC cluster deployment that the hashes matches, might be a good idea to do a fresh build and redeploy.

Contract Github releases might have been compromised, if you use the compiled WASM from the release it might be a good idea to verify it also.

The Github app token can create fake releases, could go through previous releases to see if any is odd.

(Highly speculative) Impact i am unable to verify without testing on production NEAR/MPC repo

- Any org level NEAR github secret can be extracted using same workflow vector. (Cant verify this myself if scoped to org level or repo level)
  - CARGO_REGISTRY_TOKEN
  - NEARPROTOCOL_CI_PR_ACCESS
  - NPM_TOKEN
  - MY_GITHUB_TOKEN

Using ACTIONS_ID_TOKEN_REQUEST_TOKEN and ACTIONS_ID_TOKEN_REQUEST_URL

- Any org level installed Github apps for the NEAR org that uses OIDC authentication using the ACTIONS_ID_TOKEN_REQUEST_TOKEN can be compromised. (This is how the claude code actions apps was escalated)
- Potentially possible to escalate to the GCP based nearone-sccache build cache and compromise NEARCORE builds if GCP is scoped to NEAR org level and not NEARCORE repo level, same with AWS.

nearone-sccache build cache

Target:

- WIF provider: projects/60246494556/locations/global/workloadIdentityPools/github-pool/providers/github-provider
- Service account: sccache-sa@nearone-services.iam.gserviceaccount.com
- GCS bucket: nearone-sccache, SCCACHE_GCS_RW_MODE=READ_WRITE, prefix nearcore/ci

If the GCP WIF trust policy is (attribute.repository_owner == "near") then the ACTIONS_ID_TOKEN_REQUEST_TOKEN for near/mpc can be used to retrieve sccache-sa@nearone-services credentials

This could be used to write to gs://nearone-sccache/nearcore/ci/ with a malicious .rlib for a crate. Every subsequent nearcore CI build that gets a cache hit compiles the backdoored object file into neard. This also affects any other releases that relies on this caching.

s3://build.nearprotocol.com official binary store

Target:

IAM role: arn:aws:iam::590184106962:role/GitHubActionsRunner

S3 bucket: s3://build.nearprotocol.com

Used by: neard_release.yml, neard_nightly_binary.yml, neard_assertion_binary.yml, mac_m1_binary.yml

Potential impact: Direct write to s3://build.nearprotocol.com which could overwrite contents and infect users downloading from the bucket with a backdoor

neard: nearcore/Linux/master/neard

near-workspaces: nearcore/Linux-x86_64/.../near-sandbox.tar.gz

GCP - verify that trust scope limits to near/nearcore only and not just near org:
sccache

```
gcloud iam workload-identity-pools providers describe github-provider --locati
```

clusterfuzz

```
gcloud iam workload-identity-pools providers describe github-provider --locati
```

AWS:

```
aws iam get-role --role-name GitHubActionsRunner   # account 590184106962
# Look at AssumeRolePolicyDocument.Statement[].Condition
```

Some of these issues are speculative and i could not verify because of time constraint, will look into more once this vulnerability is fixed. Anything covered by POC i am certain of, the rest is mostly speculations.

ATTACHMENTS

This report contains attachments.

- pocrepo.zip
- nearreport.md
- pocrun.mp4